Complexitate spatiala

- **o variabila de valoare maxim n → O(log n)**
- o functie – parametrii functiei nu se iau in considerare decat daca se modifica in timpul algoritmului
- functii recursive: se calculeaza cata memorie are o stare din recursivitate si care este lungimea lantului recursiv. Complexitatea este lungimea lantului * memoria pentru o stare
- la apelul recursiv, in cazul unei sume/ produs, nu se considera ca variabila ultimul apel

```
fact1(n)
r = 1
for i=1:n
        r=r*i
return r
```

Variabile:

i: maxim n -> O(log n)
r: maxim n! -> O(log n!)

complexitatea totala:O(log n + log n!) = O(log n!)

```
fact2(n)
if n ==0
        return 1
return n* fact2(n-1)
```

Variabile:

n -> O(log n)
lungimea lantului de recursivitate:

n -> n-1 -> n-2 -> n-3 ->.... 0 => n+1 apeluri care vor exista simultan pe stiva

Complexitate totala:

O((n+1) * log n) = O(n log n)

```
sum1(v,n)
s =0
for i=1:n
        s=s+v[i]
return s
```

max(v[i]) = m

Variabile:

i: maxim n -> O(log n)
s: maxim n*m -> O(log n*m)

Complexitatea totala: O(log n + log n*m) = O (log n*m)

apel: sum2(v,1,n)

```
sum2(v,i,j)
if (i == j)
        return v[i]
return sum2(v,i,(i+j)/2) + sum2(v,(i+j)/2+1, j)
```

Variabile:

i,j -> maxim n -> O(log n)

sum2(v,i,(i+j)/2) -> maxim n/2 * m -> O(log n*m)

lungimea lantului de recursivitate:

```
                n
        n/2             n/2
n/4     n/4       n/4           n/4
..
n/2^k = 1
```

=> 2^k = n => k = log n => K+1 nivele = log n+ 1 = O(log n)

Complexitatea totala:

O(log (n*m) * log n)

```
apel: sum3(v,1,n)
sum3(v,i,n)
if (i == n)
        return v[i]
return v[i]+ sum3(v,i+1,n)
```

Variabile:
i: maxim n -> O(log n)

v[i]: maxim m -> O(log m)

lungimea maxima a lantului de recursivitate:

1 -> 2 -> 3 -> ... n -> n instante ale functiei in acelasi timp

Complexitate totala:

O((log n + log m) * n) = O (log (n*m) * n)

```
N_TSP(G=(V,E), c)
V' = multimea vida
for i=1:n
        v = choice(V) // choice {1...n}
        if v apartine V'
                fail()
        V' = V' u {v}
        // V'[i] = v
V'[n+1] = V'[1]
s = 0
for i=1:n
        if (E [V'[i] ] [V' [i+1] ] == inf)
                fail()
        s = s + E [ V'[i] ] [ V' [i+1] ]

if (s <=c)
        success()
fail()
```

max(E[x][y]) = m

Variabile:

V' : o multime de n noduri. Fiecare nod dimensiune maxim n -> O(n * log n)
v: un nod de dimensiune maxim n -> O(log n)
i: maxim n -> O(log n)
s: maxim m*n -> O(log n*m)

Complexitatea totala:

O(n*log n + log n + log n*m) = O(n*log n + log n*m)